pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

SHAP Values

WEEK 3 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

What a SHAP value is, and is not

```
base value  +  64 SHAP values  =  the model's log-odds for this company
```

Allowed names

SHAP value: contribution to this row's log-odds

contribution to the model's score

mean absolute SHAP value

Never called

reason for default or reason code

cause or driver

effect of changing the feature

Exact arithmetic about the model. It describes how the output splits; it does not say why a company failed.

How SHAP values work

1. Fit the model

the SHAP values describe it

2. Every order

each column's average contribution

3. TreeExplainer

exact for trees, no sampling

4. Base + parts

equals the company's log-odds

You control

The model and the rows to explain.

What comes out

A table of SHAP values in log-odds; a description of the model on this file.

One company, sixty-four columns

-6.4588

base value (log-odds)

8.6317

sum of 64 SHAP values

2.1729

the model's log-odds

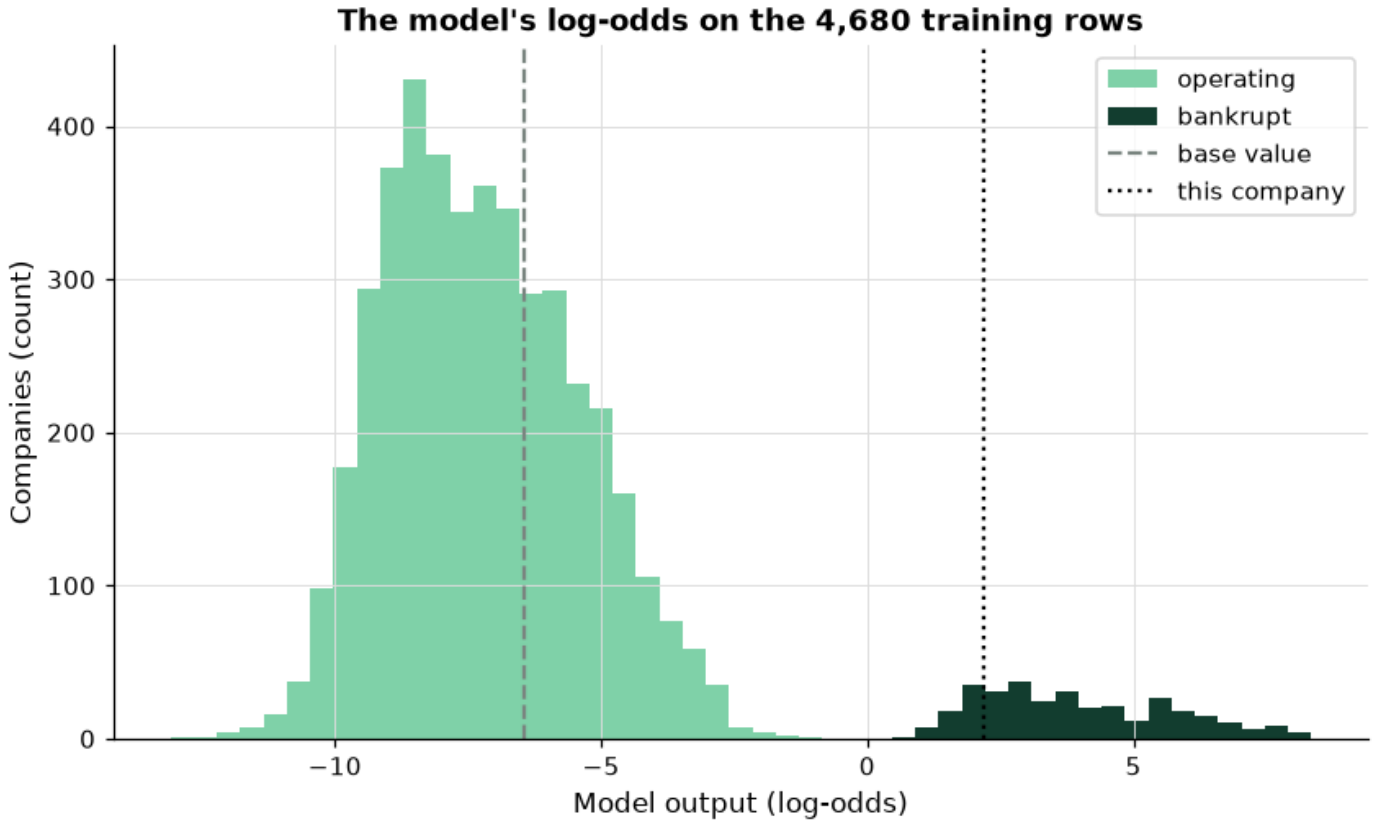
Feature	Value	SHAP value
Attr46	0.0420	2.1918
Attr27	-1.7329	0.9643
Attr38	-1.0383	0.9080
Attr35	-0.0926	0.7560
Attr39	-0.0902	0.6471
Attr58	1.1163	-0.6351

The base value

The average of the model's log-odds over the 4,680 training rows (326 bankrupt). Each SHAP value moves company 5545 away from it, and the 64 add up exactly.

LightGBM, 300 trees, learning rate 0.05, 15 leaves, fitted on Course 4's Polish training rows. No row is scored today.

Where the base value sits



Two groups, one average

2,920 of 4,680 rows sit below the base value, -6.4588. It is an average of both groups: a starting point for the arithmetic, not a typical company.

On its own rows the model puts every bankrupt company above 0 log-odds: a description of the fit, not of new companies.

How log-odds become a probability

1. Sum in log-odds

contributions add here

2. Apply the curve

p = 1 / (1 + e^-x)

3. Probability

log-odds 0 is 50 percent

4. The curve bends

log-odds add, probabilities do not

You control

Nothing: a fixed function.

What comes out

The base value and the company's output as probabilities.

SHAP values add up in log-odds, not in probability

```
mlkit.logistic_probability(2.1729) = 0.8978        base value -6.4588 -> 0.0016
```

0.0697

average model probability

0.0697

bankrupt share

0.0016

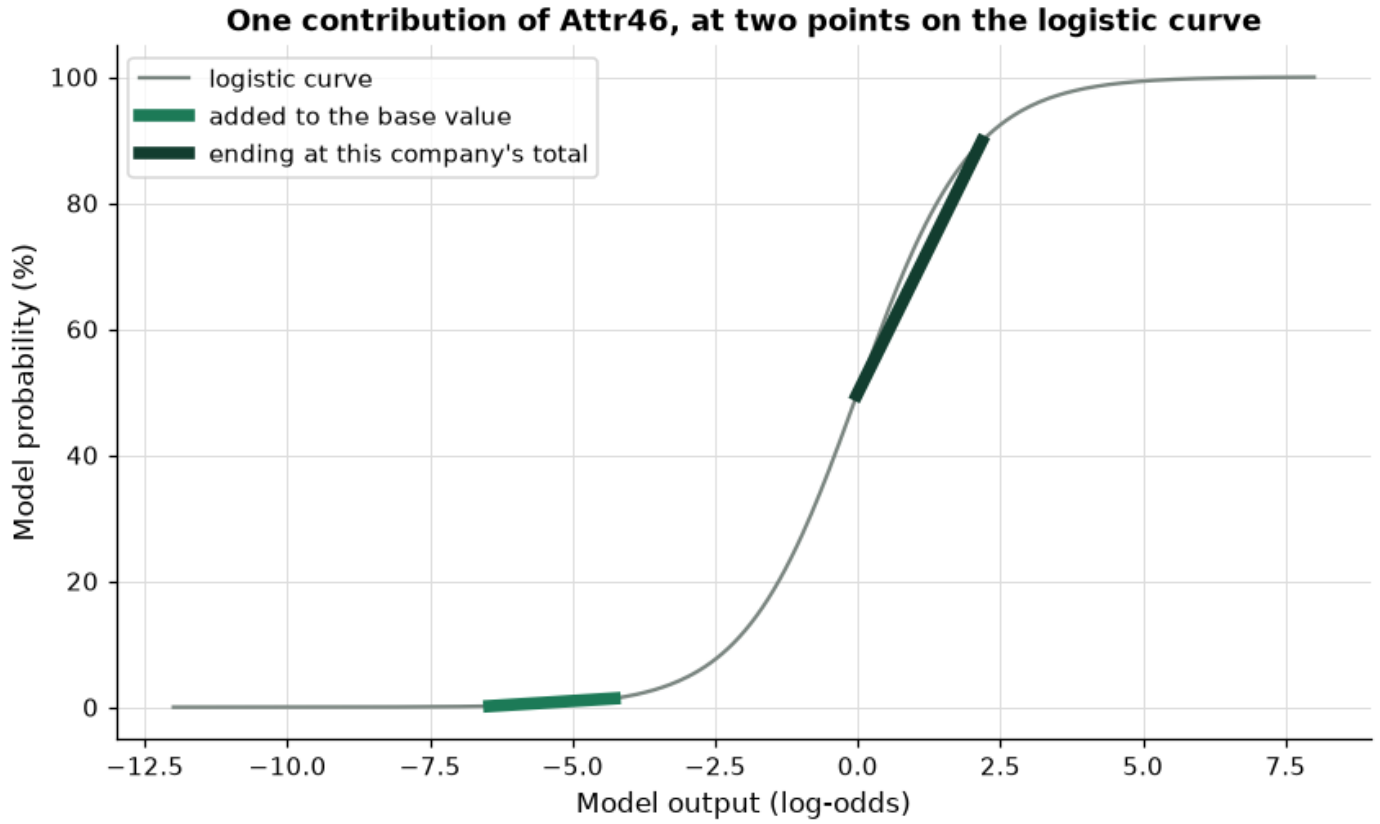
base value as a probability

The same Attr46 contribution, 2.1918 log-odds

1.23 percentage points added to the base value; 40.25 points at this company's total.

State contributions in log-odds. Turn only the total into a probability.

One contribution, two points on the curve



1.23

points at the base value

40.25

points at this company

The same Attr46 contribution, 2.1918 log-odds. Flat tail on the left, steep part on the right.

How a waterfall plot works

1. Start at base

the model's average

2. Bar per column

the longest first

3. Red up, blue down

each bar is a contribution

4. End at log-odds

the company's output

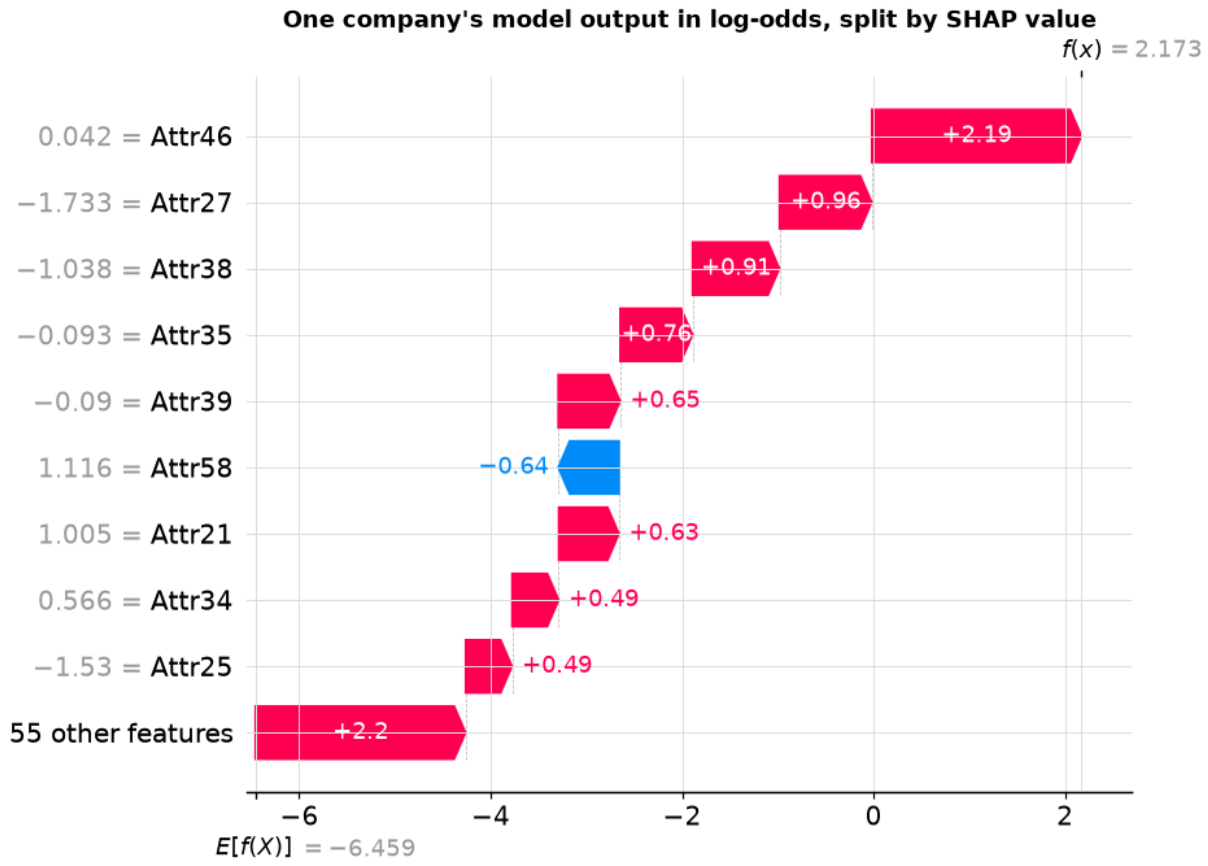
You control

The company and how many bars to show.

What comes out

One company's output split across the columns, in log-odds.

A waterfall for one company



How to read it

Bottom: E[f(X)], the base value -6.4588. One bar per feature, red up and blue down. Top: f(x), 2.1729 log-odds.

The axis is log-odds. It shows how the output is built up, not what a changed ratio would do.

How a beeswarm works

1. One dot

one company, one column

2. Position

the SHAP value, in log-odds

3. Colour

the company's value: red high, blue low

4. Order

by mean absolute SHAP value

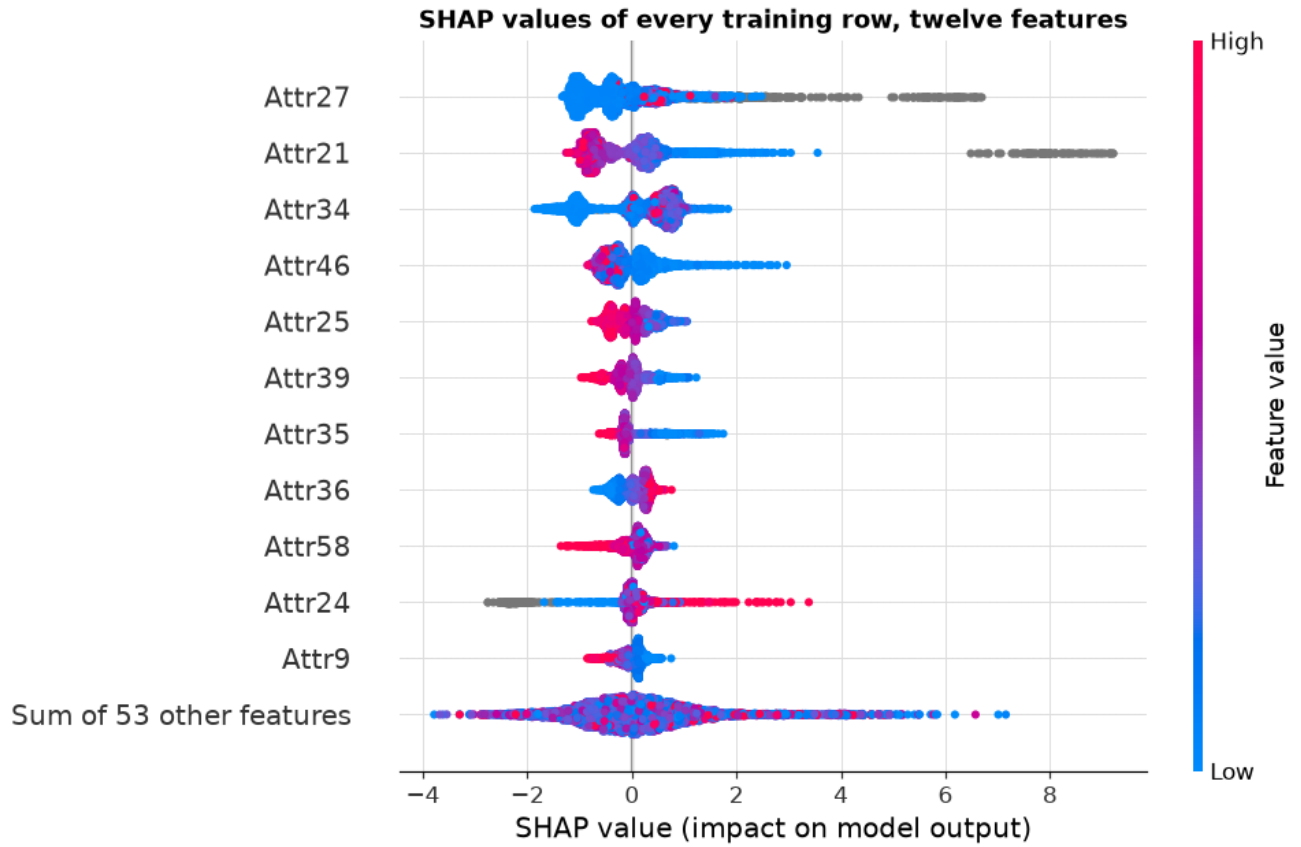
You control

The number of columns and the rows.

What comes out

A picture of how the model uses each column across the file.

The beeswarm: every company, every column



One dot per company

Position: its SHAP value for that feature. Colour: its value of the ratio, red high, blue low, grey missing.

Association in this file, in this model. Not the effect of the ratio.

A paper from May 2017

Scott Lundberg and Su-In Lee, 22 May 2017

arXiv 1705.07874

Record comment: "To appear in NIPS 2017". Last updated 25 November 2017.

Its title describes a unified approach to interpreting the outputs of models.

Source, read 5 October 2026: the arXiv record, arxiv.org/abs/1705.07874. Title, authors, dates, and comment only; the paper itself was not read for this course.

How three importance measures work

1. Mean abs SHAP

average size of the contribution

2. Gain

training loss removed by the splits

3. Split count

how often the trees split on it

4. Three orders

they can disagree

You control

The rows averaged over and how many columns to show.

What comes out

One table with the columns ranked three ways.

Mean absolute SHAP value, gain, and split counts

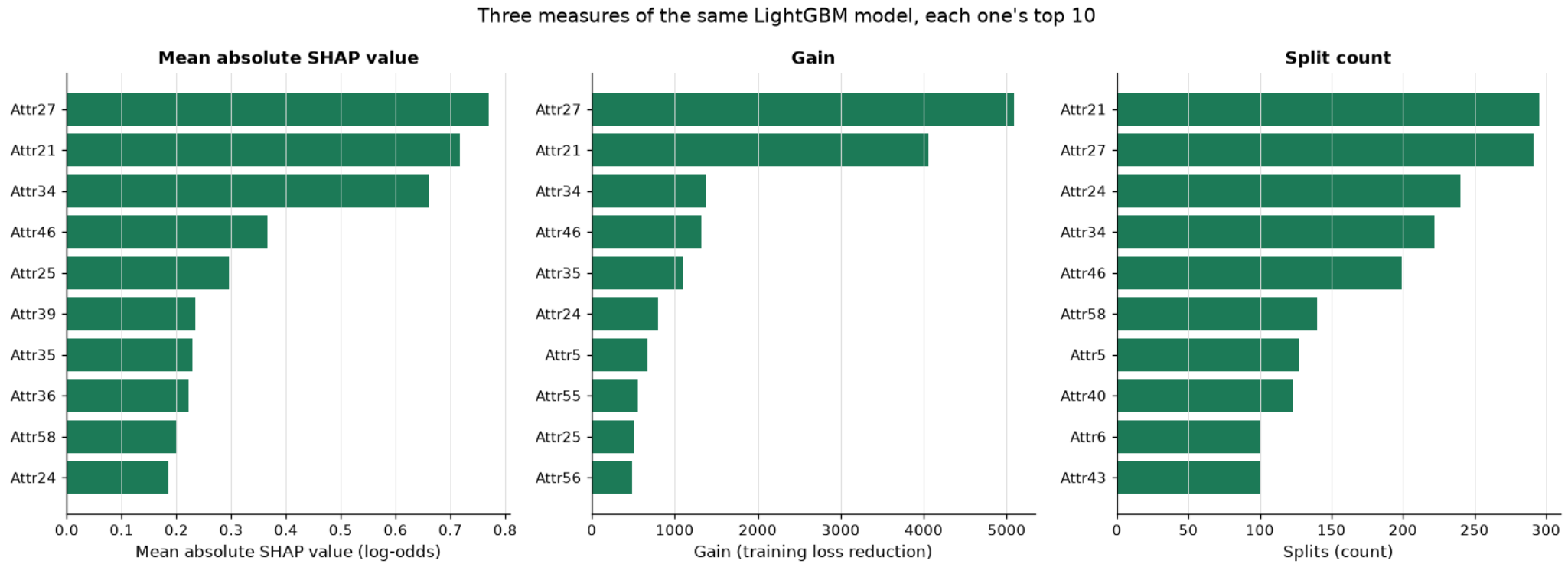
Feature	Mean abs SHAP	Rank	Gain rank	Split rank
Attr27	0.7702	1	1	2
Attr21	0.7174	2	2	1
Attr34	0.6608	3	3	4
Attr46	0.3655	4	4	5
Attr25	0.2954	5	9	12
Attr39	0.2349	6	11	14
Attr35	0.2291	7	5	17
Attr36	0.2215	8	18	15

Split counts rank differently

Top three by split count: Attr21, Attr27, Attr24. A count says how often the trees asked, not how much the answers moved the output.

All three describe this fitted model: what it leans on, an association in this file.

Three measures, drawn



SHAP and gain open with the same three bars. Split counts: Attr21, Attr27, Attr24. Compare bars within a panel, never across.

How a dependence plot works

1. x: ratio value

one column

2. y: its SHAP value

one dot per company

3. Flat: ignored

a slope or a step means it matters

4. Spread at one x

the other columns also count

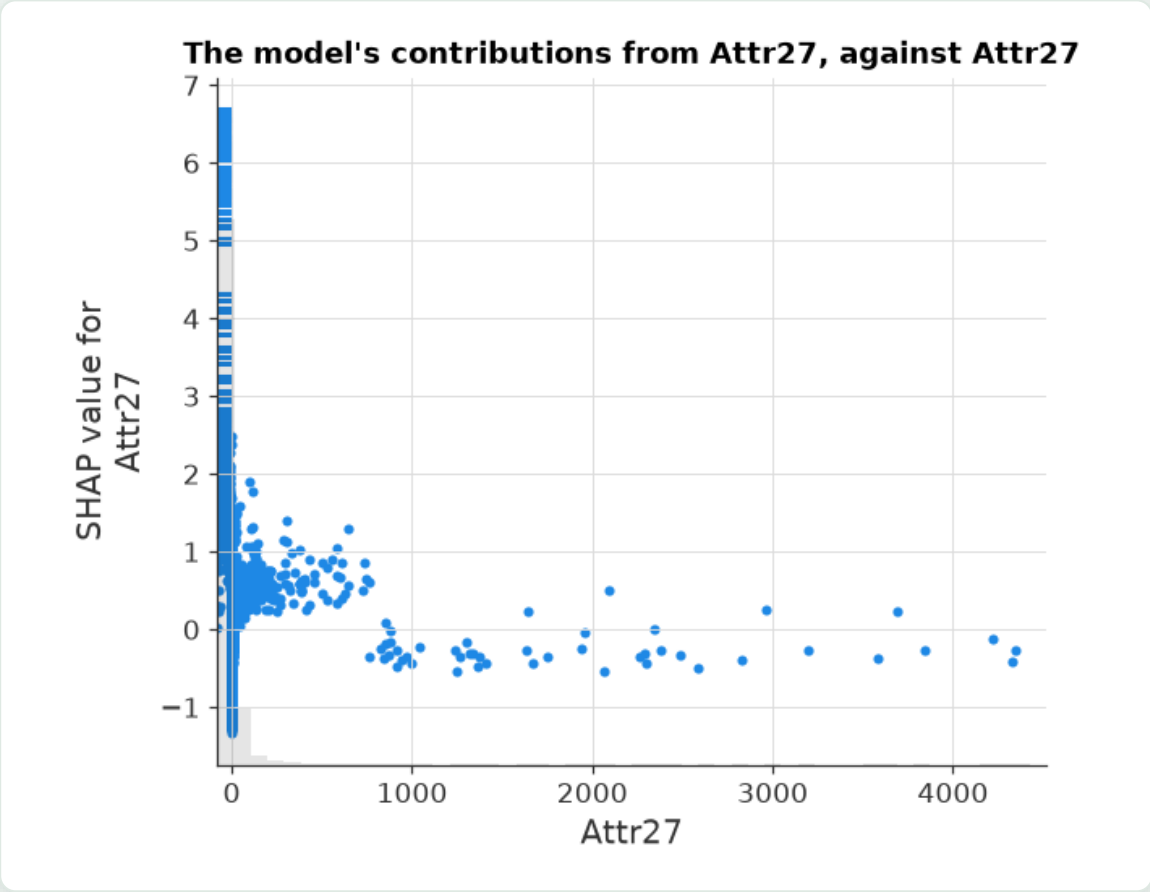
You control

The column and the percentile cut.

What comes out

A picture of the model's use of one ratio across this file's values.

The model's contributions against Attr27

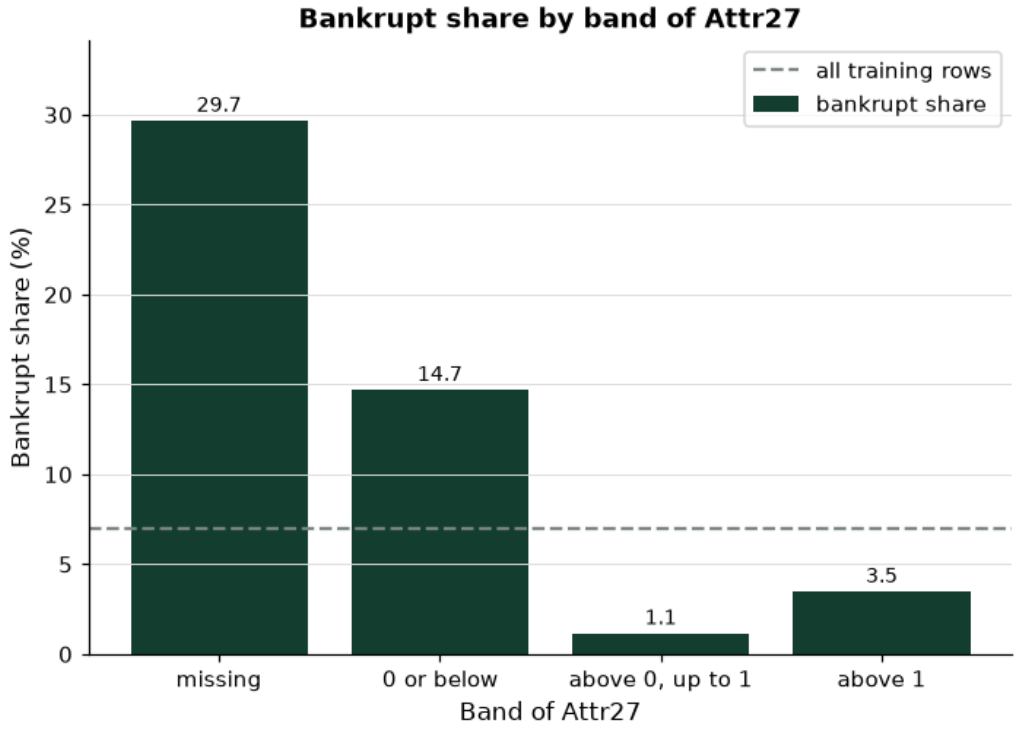


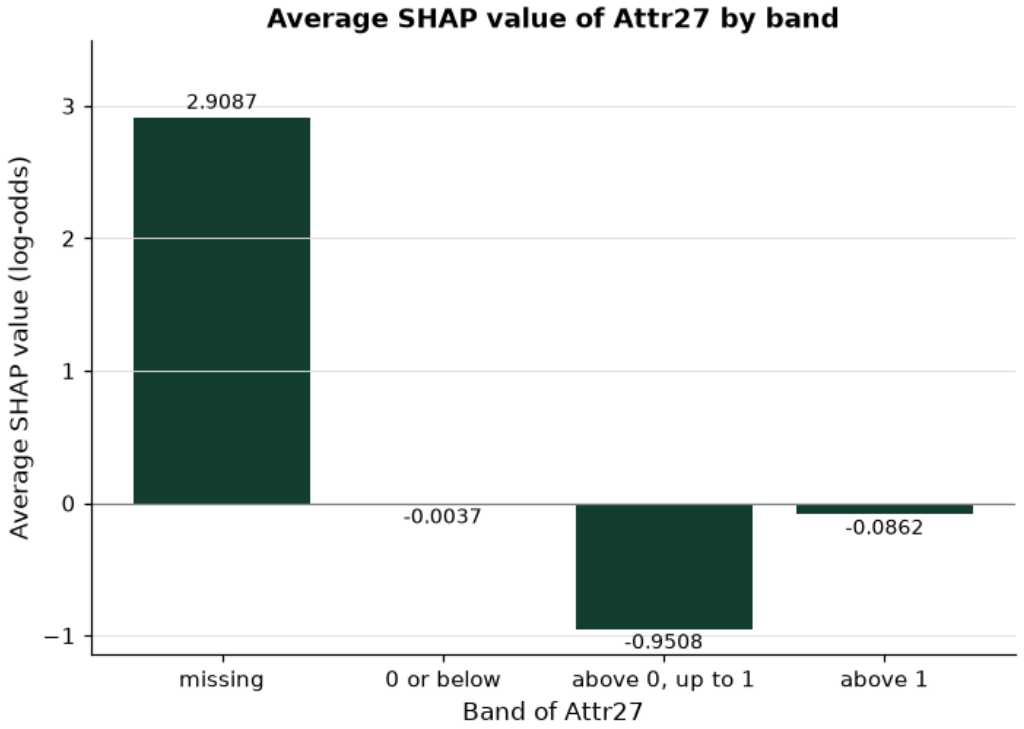
Attr27	Companies	Bankrupt	Avg SHAP
missing	317	0.2965	2.9087
0 or below	974	0.1468	-0.0037
above 0, up to 1	1225	0.0114	-0.9508
above 1	2164	0.0347	-0.0862

X27: profit on operating activities / financial expenses. Missing gets the largest contributions; 0 or below sits near zero despite a high bankrupt share.

Described, never read as the effect of changing the ratio.

The bands, drawn





0 or below: bankrupt share 0.1468, average SHAP value -0.0037. Tall on the left, almost flat on the right.

How SHAP shares between twin columns

1. Twin columns

a ratio and its reciprocal

2. Either one splits

the trees ask the same question

3. SHAP shares

the division depends on the splits

4. Read the pair

add their SHAP values

You control

Which columns to compare.

What comes out

Their correlation, and each SHAP value beside the sum.

Two columns, one question: Attr17 = 1 / Attr2

4,665 of 4,666

rows: Attr17 = 1 / Attr2

-0.9986

rank correlation

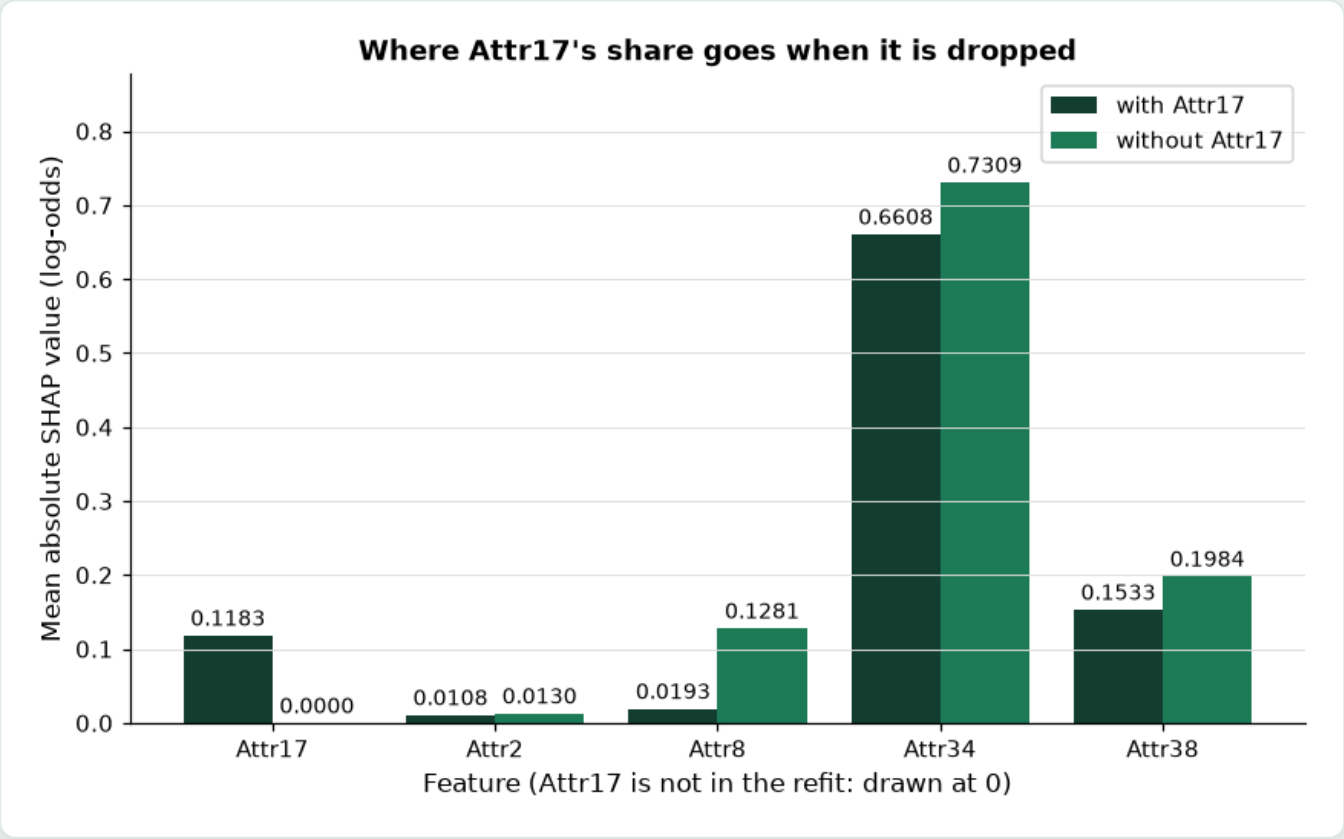
Feature	Mean abs SHAP	Rank
Attr17	0.1183	22
Attr2	0.0108	60
Attr2, Attr17 dropped	0.0130	

Without Attr17, the largest rises

```
Attr8 +0.1088;  Attr34 +0.0701;  Attr38 +0.0451
```

SHAP credits the column the trees used. A ranking describes one model with one set of columns.

Where Attr17's share goes



Feature	With	Without	Change
Attr17	0.1183	dropped	
Attr2	0.0108	0.0130	0.0022
Attr8	0.0193	0.1281	0.1088
Attr34	0.6608	0.7309	0.0701
Attr38	0.1533	0.1984	0.0451

Mean absolute SHAP value, log-odds. The twin barely moves; the share spreads to other columns.

How XGBoost's SHAP values differ

1. Same explainer

TreeExplainer works on XGBoost

2. Float32 trees

XGBoost stores 32-bit numbers

3. Looser check

agreement to about 1e-5, not 1e-9

4. Same property

SHAP values still add up

You control

The tolerance of the additivity check.

What comes out

The same table, to the precision the library stores.

XGBoost's SHAP values, in float32

float32

SHAP values stored as

False

add up within 1e-9

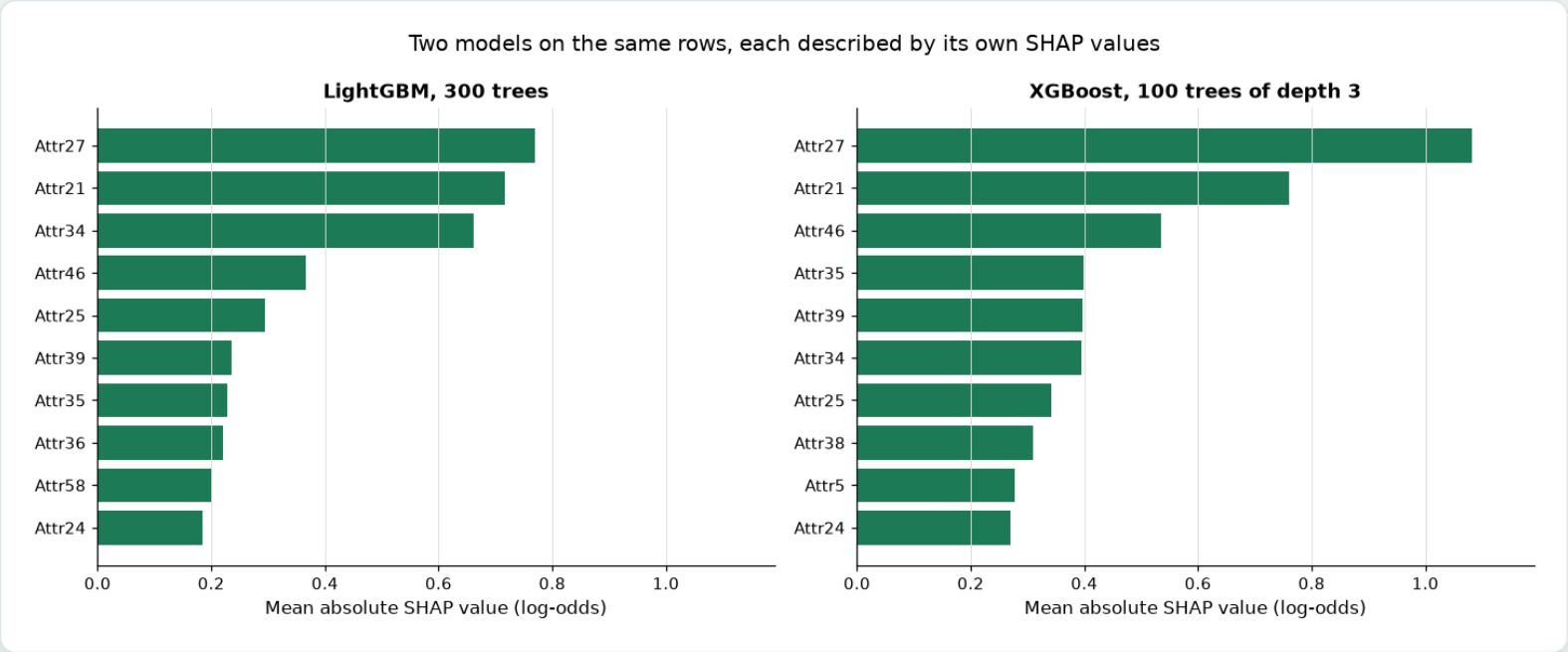
True

add up within 1e-5

```
ensemblekit.assert_additive(shap_values, raw, tol=1e-9)       # LightGBM, float64
ensemblekit.assert_additive(xgb_values, xgb_margin, tol=1e-5)  # XGBoost, float32
```

About seven significant digits: no XGBoost output is printed beyond 4 decimals.

Two models, two descriptions



0.7702

Attr27, LightGBM

1.0829

Attr27, XGBoost

One shared scale. Each panel describes its own model only.

The words: assert_described

Allowed names

SHAP value: contribution to this row's log-odds

contribution to the model's score

mean absolute SHAP value

Never called

reason for default or reason code

cause or driver

effect of changing the feature

```
EXPLANATION_WORDS = ['cause', 'causes', 'caused', 'driver', 'drivers', 'drives', 'reason', 'reasons', 'because']
```

A reminder, not a proof. A description that needs "because" is rewritten.

Excel side by side

Excel side by side (tested in Excel)

`=B1+SUM(B2:B65)`	the company's log-odds, 2.1729
`=1/(1+EXP(-B66))`	its probability, 0.8978
`=1/(1+EXP(-B1))`	base value as a probability, 0.0016
`=AVERAGE(ABS(D2:D4681))`	Attr27 mean absolute SHAP, 0.7702
`=AVERAGE(E2:E4681)`	the base value, -6.4588
`=AVERAGEIFS(D2:D4681,A2:A4681,">1")`	Attr27 above 1, average SHAP -0.0862

No SHAP function in Excel: the values come from walking every tree. Excel checks what they add up to.

Save the day: a tag

1. Commit

mlkit.py, ensemblekit and its tests

2. Log

shap_table.csv, top 10, 4 decimals

3. Tag

git tag week3-teaching

4. Read

git log --oneline --decorate

```
model,feature,mean_abs_shap,rank
lightgbm_300_trees,Attr27,0.7702,1
lightgbm_300_trees,Attr21,0.7174,2
lightgbm_300_trees,Attr34,0.6608,3
lightgbm_300_trees,Attr46,0.3655,4
```

A tag names a commit, so the last teaching day of week 3 is found by name. 56 tests pass at the end of today.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: base value plus SHAP values is read as a probability and clipped: 1.0000 for a company the model gives 0.8978.

Code that runs is not the same as code that is right.

What you can do now

- Base value plus a row's SHAP values = the model's log-odds
- Contributions in log-odds; only the total becomes a probability
- Waterfall for one company, beeswarm for every company
- Mean absolute SHAP, gain, and split counts rank differently
- A SHAP value describes the model, never a cause

Week 3, Day 5 The capstone: a new file, four models, one description